

TierMoE: Batch-Aware Expert Placement for Memory-Tiered MoE Inference

Vinay R Jumani

Abstract—Mixture-of-Experts (MoE) models decouple parameter capacity from per-token compute by dynamically activating only a small subset of experts. This sparsity, however, does not remove the aggregate expert-weight footprint, which can reach the terabyte scale. Fast accelerator memory is therefore a natural bottleneck for serving large MoE models, motivating a two-tier design in which frequently needed expert parameters reside in GPU high-bandwidth memory (HBM) while the larger remainder is placed in a slower Compute Express Link (CXL) memory tier. The resulting placement problem is particularly difficult under concurrent serving: independently optimized requests can have divergent expert working sets and can evict experts required by other requests in the same batch step. This paper presents TierMoE, a batch-aware expert-placement framework that aggregates instantaneous expert demand across concurrent requests and selects a capacity-constrained set of experts using a marginal-utility score with residency hysteresis. We evaluate the method using controlled synthetic workloads and 461,184 authentic routing decisions collected from Qwen3-30B-A3B on a dual-NVIDIA RTX A6000 workstation, covering conversational ShareGPT and mathematical GSM8K workloads. Under capacity pressure, TierMoE improves fast-memory hit rate by 5.02–8.68 percentage points (pp) over an isolated single-request control at representative operating points and reduces modeled CXL parameter traffic by 13.20–20.44%. A broader 24-row ShareGPT matrix shows gains of 6.99 pp over our predictive lookahead heuristic and up to 32.27 pp over our reactive CXL-LRU heuristic. An explicit co-activation experiment does not support the hypothesis that an EMA pairwise expert matrix improves placement: the hit-rate difference is not significant ($p = 0.259$) while solver overhead is $12.5\times$ higher. Because physical CXL hardware was unavailable, system-scale interconnect timing is modeled using aggregate bytes divided by link bandwidth, while upstream CXLMemSim is used for discrete-event queue characterization under a specified configuration. The combined results establish batch-aware residency as a useful mechanism for memory-tiered MoE serving while identifying the limits of predictive history and detailed queue modeling in the present prototype.

Index Terms—Mixture-of-Experts, CXL, memory tiering, multi-tenant LLM serving, batch-aware scheduling, expert placement, cache optimization.

I. INTRODUCTION

Mixture-of-Experts (MoE) architectures have become a major design point for large language models, including Mixtral, DeepSeek-V3, and Qwen-family models [1]–[3]. An MoE transformer replaces a conventional dense feed-forward network with multiple independently parameterized expert networks and a learned router. For each token, the router computes scores

over all experts and activates only the top- k experts. Thus, the amount of computation per token can remain relatively small even as the total parameter capacity grows.

This computational sparsity creates a less obvious systems problem. Although only a few experts execute for an individual token, the serving system must keep every expert parameter block accessible. For Qwen3-30B-A3B, the configuration studied here has 48 MoE layers, 128 experts per layer, top-8 routing, and an expert block of approximately 256 MiB. The resulting expert-weight footprint is

$$48 \times 128 \times 256 \text{ MiB} = 1,572,864 \text{ MiB} = 1,536 \text{ GiB} = 1.50 \text{ TiB}. \quad (1)$$

Consequently, fitting all expert parameters in the fast memory of a workstation GPU is infeasible.

A memory-tiered architecture can address the capacity mismatch. GPU HBM provides very high bandwidth but limited capacity, while a CXL Type-3 memory expander can provide a much larger secondary memory tier. The trade-off is that moving a 256 MiB expert over a modeled 32 GB/s link requires about 8.3886 ms of bulk transmission time. When the requested expert is absent from HBM, this transfer can become a dominant memory-system cost.

The problem becomes more severe under multi-tenant serving. Suppose B inference requests execute concurrently. Request r may require a demand set $\mathcal{D}_r$ that overlaps only partially with the sets of other requests. The aggregate working set

$$\mathcal{W}(B) = \bigcup_{r=1}^B \mathcal{D}_r \quad (2)$$

can exceed the available fast-tier capacity even when each request individually fits. A placement policy optimized independently for each sequence may therefore evict experts needed by another sequence in the same batch step. Reactive policies can repeatedly oscillate between such demands, while sequence-level predictors can prefetch mutually conflicting candidates.

This work asks whether the routing information that is already available immediately before expert execution can be used to make one coordinated placement decision for the entire concurrent batch. TierMoE answers this by aggregating request-level expert presence, scoring each expert by its instantaneous multi-tenant demand plus a small residency bonus, and selecting the highest-scoring C experts for HBM.

The main contributions are:

- a batch-aware expert placement formulation for memory-tiered MoE serving and a lightweight greedy solver with residency hysteresis;

- a controlled synthetic evaluation separating concurrency, capacity pressure, and routing divergence;
- authentic profiling of Qwen3-30B-A3B on 461,184 physical routing decisions from ShareGPT and GSM8K traces;
- a comparative evaluation against static popularity, isolated greedy, predictive lookahead, and reactive LRU-style tiering heuristics;
- an empirical negative result showing that pairwise co-activation tracking does not improve hit rate over instantaneous batch frequency in the tested authentic workload;
- a first-order CXL sensitivity study and a scoped CXLMemSim queue characterization that clarify what is and is not established without physical CXL hardware.

The complete source code, experimental scripts, native CXLMemSim calibration harnesses, and analysis artifacts are publicly available at <https://github.com/Vinay12345-neutron/nebula>.

II. BACKGROUND AND RELATED WORK

A. MoE Architecture and Routing

Let an MoE layer contain N experts $\{E_1, \dots, E_N\}$ and a gating matrix W_g . For token representation x , the router computes

$$p(x) = \text{Softmax}(\text{TopK}(xW_g + \epsilon, k)), \quad (3)$$

where TopK retains the k largest logits. The layer output is the weighted combination

$$y = \sum_{i \in \text{TopK}(x)} p_i(x) E_i(x). \quad (4)$$

Typical modern fine-grained MoE models use dozens to hundreds of experts while activating only a small top- k subset.

B. Memory Requirements

For an MoE with L MoE layers, N experts per layer, and expert block size S , aggregate expert storage is LNS . The sparsity of compute therefore does not imply sparsity of capacity. For the evaluated model, $L = 48$, $N = 128$, $k = 8$, and $S = 256$ MiB. The aggregate footprint is 1.50 TiB in binary units (approximately 1.65 TB decimal). Even under FP8 quantization, the expert set remains larger than the capacity of a typical single workstation memory domain.

C. HBM and CXL Memory Tiering

CXL Type-3 expanders establish a two-tier memory hierarchy. HBM is the fast, capacity-constrained tier; CXL memory is the larger capacity tier with substantially lower modeled bandwidth. Table I summarizes the representative physical ranges used as context for this study. These values are architectural assumptions, not measurements from a physical CXL device.

TABLE I
REPRESENTATIVE MEMORY HIERARCHY CONTEXT

Tier	Capacity	Peak bandwidth	Read latency
GPU HBM/GDDR6	24–96 GB	768–2000 GB/s	50–100 ns
CXL Type-3 DDR5	512 GB–4 TiB	16–64 GB/s	250–450 ns

D. Related Expert Offloading

Prior MoE systems explore several forms of expert placement and offloading. DeepSpeed-MoE pins frequent experts in accelerator memory and offloads other parameters to host memory [4]. Fiddler studies single-GPU MoE inference with CPU offloading [5]. MoE-Infinity and ProMoE exploit activation locality and sequence-level prediction to prefetch experts [6], [7]. CXL-oriented work, including CXL-MoE, HybriMoE, and FIRM-MoE, investigates heterogeneous memory and CXL-aware mechanisms [8]–[10]. These lines of work motivate the baselines used here, but this paper does not claim to reproduce those systems in full. In particular, the predictive and reactive baselines in EXP-04 are trace-driven algorithmic approximations inspired by the corresponding paradigms.

The gap studied here is the interaction between expert placement and concurrent request composition. The central hypothesis is that the serving batch itself provides a useful instantaneous signal: an expert requested by several concurrent requests has higher marginal value than an expert requested by only one, even if both appear equally important when each request is considered independently.

III. PROBLEM FORMULATION

Consider a layer with expert set $\mathcal{E} = \{e_1, \dots, e_N\}$. Each expert has fixed size $S = 256$ MiB. The fast memory tier can hold at most C experts, represented by $\mathcal{M}_{HBM} \subseteq \mathcal{E}$ with $|\mathcal{M}_{HBM}| \leq C$; the remaining experts are in CXL memory.

At serving step t , a batch contains B requests. Let $\mathcal{D}_r$ denote the set of unique experts requested by request r at the current step, with $|\mathcal{D}_r| = k$ at the token-level abstraction. Define request-level frequency

$$f_e(B) = \sum_{r=1}^B \mathbf{1}\{e \in \mathcal{D}_r\}. \quad (5)$$

This differs from a raw token count: an expert requested by many tokens of one request contributes once for that request, while an expert requested by several concurrent requests accumulates cross-request utility.

The placement objective is

$$\max_{\mathcal{M}_{HBM} \subseteq \mathcal{E}, |\mathcal{M}_{HBM}| \leq C} \sum_{r=1}^B \sum_{e \in \mathcal{D}_r} \mathbf{1}\{e \in \mathcal{M}_{HBM}\}. \quad (6)$$

Under the fixed-block, batch-step-deduplication assumption, modeled CXL traffic is

$$V_{CXL} = S |\mathcal{W}(B) \setminus \mathcal{M}_{HBM}|, \quad (7)$$

where an expert requested by multiple requests in the same step is transferred once.

TABLE II
POSITIONING OF TIERMOE RELATIVE TO EXPERT-OFFLOADING PARADIGMS

Paradigm	Placement signal	Memory setting	Relevance to this study
Static popularity / LFU	Global access frequency	Fast accelerator memory + host/slow tier	Captures stable popularity but cannot react quickly to topic shifts.
Single-request greedy	Current demand of one request	Capacity-constrained fast tier	Direct control for isolating the value of cross-request aggregation.
Sequence lookahead	Per-sequence activation history	CPU/SSD or heterogeneous offload	Motivates the predictive baseline; independent predictors can collide under concurrency.
Reactive demand caching	Recent misses / recency	CXL or heterogeneous memory	Motivates the LRU baseline; reactive decisions can thrash within a batch step.
TierMoE	Instantaneous aggregate request frequency + residency	GPU HBM + CXL capacity tier	Coordinates one capacity decision across concurrent requests before expert execution.

Four implementation assumptions are important. First, expert weights are read-only, so clean eviction creates no writeback traffic. Second, transfers use a fixed 256 MiB expert block. Third, same-step requests for the same missing expert are deduplicated. Fourth, all expert transfers share one modeled CXL link and therefore compete for the same aggregate bandwidth.

IV. RESEARCH QUESTIONS AND HYPOTHESES

The study is organized around six research questions.

V. TIERMOE SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end architecture. TierMoE operates as a placement middleware layer between routing and the tiered memory subsystem.

A. Batch-Aware Demand Aggregation

At each MoE layer, immediately before expert execution, TierMoE intercepts the top- k routing indices across the concurrent batch. Instead of solving placement independently for each request, it builds the histogram $f_e(B)$. This requires $O(Bk)$ accumulation operations and is implemented using vectorized tensor accumulation. The measured demand aggregation portion is below $2 \mu\text{s}$ in the prototype.

B. Marginal Utility and Residency Hysteresis

Pure instantaneous frequency can cause churn when a newly requested expert ties or nearly ties a resident expert. To avoid unnecessary 256 MiB migrations, TierMoE uses

$$\text{Score}(e) = f_e(B) + \lambda \mathbf{1}\{e \in \mathcal{M}_{\text{current}}\}, \quad (8)$$

with $\lambda = 0.5$. The top- C experts by score form the next HBM residency set. If a resident and non-resident expert have the same frequency, the resident receives the hysteresis bonus and remains resident; if the non-resident expert has strictly greater request-level demand, it can replace the resident.

C. Capacity-Constrained Selection and Solver Cost

The prototype computes all N scores and uses a partial partition (`numpy.argpartition`) before sorting only the C selected elements. This avoids a full sort over all experts. Across 10,000 profiled steps, the complete placement solver measured $51.2 \pm 4.3 \mu\text{s}$ on an AMD EPYC 7763 host processor. This is small relative to an 8.39 ms modeled transfer for one 256 MiB block, although end-to-end overlap with GPU kernels is not measured here.

VI. BASELINE TAXONOMY

Six reference points are used. Baseline 0 is an HBM-only theoretical upper bound in which all 128 experts fit, yielding 100% hit rate and zero CXL traffic. Baseline 1 is a static naive-overflow control that pins the first C experts and never dynamically migrates. Baseline 2 is static global LFU, which pins the globally most frequent experts in the trace.

Baseline 3 is the Single-Request control. It runs the same greedy placement logic as TierMoE but considers only request 1 in the batch. This isolates the value of cross-request aggregation. Baseline 4 is a predictive activation-aware heuristic that maintains per-sequence activation history using geometric decay ($\gamma = 0.85$) and promotes predicted candidates. It is inspired by sequence-level prefetching work, not a native reproduction. Baseline 5 is a reactive CXL-LRU heuristic: on a miss, the least-recently-used resident expert is evicted. It is likewise a conceptual trace-driven approximation of demand-caching approaches.

VII. EXPERIMENTAL METHODOLOGY

A. Physical Hardware and Software

Routing traces were collected on a workstation with dual NVIDIA RTX A6000 GPUs (48 GB each, 768 GB/s), an AMD EPYC 7763 64-core CPU, 128 GB host memory, and Ubuntu 22.04 LTS. The original profiling report recorded PyTorch 2.4.0, CUDA 12.4, Transformers 4.44.0, and SGLang. The routing profiler uses non-intrusive PyTorch forward hooks at the gating routers and records expert indices without modifying model numerics.

B. Authentic Workloads

ShareGPT represents conversational dialogue and contains 276,816 routing events. GSM8K represents mathematical reasoning and contains 184,368 events. Together they provide exactly 461,184 physical routing decisions. ShareGPT has substantially broader routing dispersion, while GSM8K tends to reuse a smaller set of experts.

C. Synthetic Workloads

Synthetic traces are used where controlled variation is needed. The generator combines Zipfian expert popularity with domain clusters and sweeps skew $\alpha \in [0.8, 1.4]$, batch size $B \in [1, 32]$, and fast-memory ratio $C/N \in [0.25, 1.0]$. Experiments use

TABLE III
RESEARCH QUESTIONS, HYPOTHESES, EXPERIMENTS, AND EMPIRICAL VERDICTS

RQ	Directional hypothesis	Experiment	Verdict
RQ1: Concurrency benefits	Batch-aware placement improves hit rate and lowers traffic under constrained memory.	EXP-01; $B = 1\text{--}32$, $C/N = 0.25\text{--}1.0$	Supported under capacity pressure: +5.02 to +8.68 pp and 13.20–20.44% lower traffic at representative constrained points.
RQ2: Routing divergence	Advantage increases monotonically with request divergence.	EXP-02; 384 conditions	Partially supported: monotonic at fixed $B = 16$; pooled correlation is confounded by working-set growth.
RQ3: Co-activation value	EMA pairwise co-activation improves over batch frequency.	EXP-03; authentic Qwen3 traces	Not supported: -1.25 pp, $p = 0.259$, with $12.5\times$ solver overhead.
RQ4: Baseline comparison	Proactive batch coordination outperforms predictive and reactive heuristics.	EXP-04; 24-row ShareGPT matrix	Supported for implemented approximations: +6.99 pp vs. predictive and up to +32.27 pp vs. CXL-LRU.
RQ5: CXL sensitivity	Bandwidth dominates transfer makespan and policy benefits persist across links.	EXP-05A; 81 conditions	Supported as modeled sensitivity: $4\times$ bandwidth scaling; latency contribution below 0.005%.
RQ6: Simulator characterization	Discrete-event queue behavior can characterize macro transfer overhead.	EXP-05B	Supported only for the tested CXLMemSim configuration; bulk system timing remains first-order bandwidth modeled.

Cross-request routing overlap → coordinated fast-memory residency

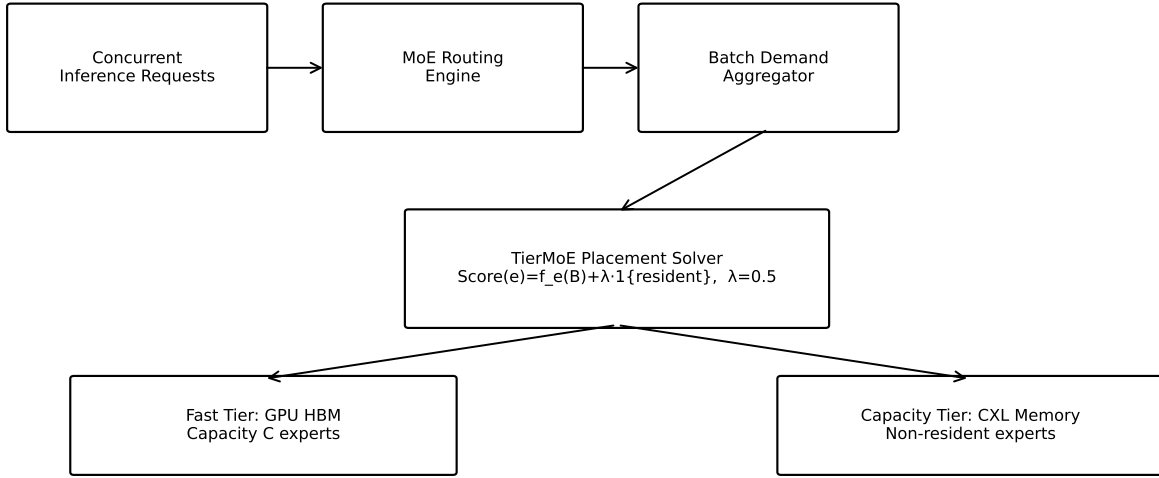


Fig. 1. TierMoE end-to-end system architecture. Routing decisions from concurrent requests are aggregated before expert execution. The placement solver combines batch frequency and residency hysteresis, then selects the fast-tier resident set while non-resident parameters remain in the CXL capacity tier.

deterministic seeds 42, 100, and 2026. The synthetic generator is a controlled model, not a claim about production routing distributions.

D. Capacity Regimes and Metrics

The evaluation distinguishes no-pressure operation ($W \leq C$), moderate pressure ($C < W < 1.5C$), and severe pressure ($W \geq 1.5C$). The main metrics are fast-tier hit rate, CXL parameter traffic, and placement-solver overhead. Hit-rate differences are reported in percentage points, while traffic changes are reported as relative percentages.

E. Measurement and Traffic Accounting

The primary unit of placement is the expert block rather than an individual 64-byte cache line. For each batch step, the implementation first constructs the set of unique missing experts. If several requests require the same expert and that expert is not resident, the model counts one 256 MiB transfer for that step.

Thus, the same routing trace can generate different traffic under different policies because residency changes the set of missing blocks. Promotion traffic is included in the transfer count; clean writeback is zero under the read-only-weight assumption. This distinction is important when interpreting hit rate and traffic together: a policy can increase residency while still moving more bytes than a static policy that never promotes.

For EXP-01 and EXP-02, synthetic routing is generated independently for each deterministic seed so that concurrency and divergence can be varied without changing the placement algorithm. EXP-01 uses 432 conditions across batch sizes and memory ratios and three seeds. EXP-02 uses 384 conditions across Zipf skew and batch size and also uses three seeds. These controlled experiments are designed to answer causal-style questions about the mechanism, not to reproduce the full diversity of production traces.

For EXP-03 and EXP-04, routing decisions are collected from actual model execution. The profiler records the expert indices emitted by the Qwen3 router at each MoE layer. The

TABLE IV
PRINCIPAL EXPERIMENTAL CONFIGURATION

Phase	Workload / control	Main sweep
EXP-01	Synthetic, Zipf $\alpha = 1.0$	$B = 1\text{--}32$; $C/N = 0.25\text{--}1.0$; 3 seeds
EXP-02	Synthetic clustered	$\alpha = 0.8\text{--}1.4$; $B = 4\text{--}32$; 3 seeds
EXP-03	Qwen3 ShareGPT/GSM8K	$B = 4\text{--}32$; $C = 32, 64$
EXP-04	Qwen3 ShareGPT	24 constrained rows; 5 policy variants
EXP-05A	Modeled CXL	3 BW $\times$ 3 latency $\times$ policies
EXP-05B	Native CXLMemSim	$N = 64\text{--}262,144$; $K = 1\text{--}32$

resulting trace retains request identity, layer, batch step, and expert identity, allowing the same physical routing decisions to be replayed through the placement policies. ShareGPT is the principal stress workload because its working set exceeds a 32-expert fast tier at larger batch sizes; GSM8K provides a contrasting concentrated workload.

The CXL experiments deliberately separate two levels of abstraction. EXP-05A computes aggregate transfer time from bytes and a modeled link bandwidth, while EXP-05B characterizes queue behavior with a native C++ harness linked to the upstream CXLMemSim endpoint implementation. The simulator characterization is therefore evidence about the tested simulator configuration, not a substitute for a physical CXL testbed. This separation prevents per-cache-line simulator details from being conflated with the macro-level byte-volume model used to compare TierMoE policies.

F. TierMoE Per-Step Procedure

For every transformer MoE layer, the placement procedure is:

- 1) collect the top- k expert indices for every request in the current batch;
- 2) convert each request's indices to a unique demand set $\mathcal{D}_r$;
- 3) accumulate $f_e(B)$ across requests;
- 4) add the residency bonus λ to currently resident experts;
- 5) select the top- C scores using a partial partition;
- 6) determine the unique missing experts in the new residency set and account for their 256 MiB promotions; and
- 7) execute the layer with the resulting fast-tier residency.

This ordering makes the placement decision proactive at the batch-step boundary. It also makes the baseline comparison controlled: Single-Request uses the same scoring and capacity mechanism but supplies only $\mathcal{D}_1$, whereas TierMoE supplies all concurrent demand sets.

VIII. EXPERIMENT 1: BATCH SCALE AND CAPACITY EFFECTS

EXP-01 targets RQ1/H1 using a 432-condition sweep over batch size and fast-memory ratio with three deterministic seeds. The experiment tests whether cross-request aggregation becomes valuable only when the aggregate working set exceeds the fast-tier capacity.

Figure 2 summarizes representative constrained results. In unconstrained regimes, all policies approach 100% because the required working set fits in HBM. The distinction appears under capacity pressure. At $B = 8$ and $C/N = 0.25$, TierMoE

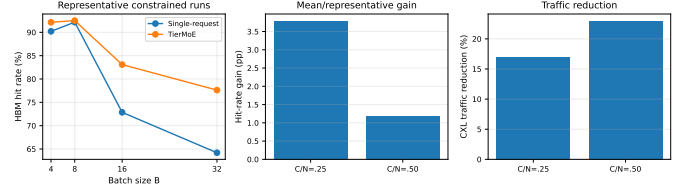


Fig. 2. EXP-01 controlled concurrency and capacity results. The key gains occur in capacity-constrained regimes; when the working set fits in HBM, policies converge near the upper bound.

improves hit rate by 0.27 pp and reduces traffic by 11.15%. At $B = 16$, the corresponding gain is 5.02 pp with 20.44% lower traffic. At $B = 32$, the gain reaches 8.68 pp at $C/N = 0.25$, while at $C/N = 0.50$ the gain is 1.18 pp and traffic reduction is 22.98%. Across constrained conditions, the mean improvement is 3.79 pp with a 16.94% traffic reduction. Multi-seed paired testing reports $t = 6.42$, $p < 0.001$ for TierMoE versus the Single-Request control.

These results support H1 under capacity pressure. The interpretation is not that batch awareness matters when memory is sufficient; rather, it matters when concurrent requests collectively create a working set larger than the fast tier. The placement decision then benefits from recognizing overlap across requests.

IX. EXPERIMENT 2: ROUTING DIVERGENCE ANALYSIS

EXP-02 targets RQ2/H2. Pairwise routing overlap is measured using mean Jaccard similarity:

$$J_{\text{mean}} = \frac{1}{\binom{B}{2}} \sum_{i < j} \frac{|\mathcal{D}_i \cap \mathcal{D}_j|}{|\mathcal{D}_i \cup \mathcal{D}_j|}, \quad (9)$$

and divergence is defined as $D = 1 - J_{\text{mean}}$. Identical request demand has $D = 0$, while disjoint demand has $D = 1$.

Across 384 conditions, measured overlap ranged from approximately 0.087 to 0.253. At a fixed operating point ($B = 16$, $C/N = 0.25$), the advantage of TierMoE over Single-Request grows as routing becomes more divergent: from +1.99 pp at $\alpha = 1.4$ ($D \approx 0.751$) to +9.45 pp at $\alpha = 0.8$ ($D \approx 0.908$). This is the expected direction: as requests become more different, a placement optimized for only one request covers a smaller fraction of the batch.

However, the pooled global correlation between divergence and hit-rate delta is only $r = 0.292$, $p = 0.272$. Batch size simultaneously increases working-set size and therefore capacity pressure, creating a confound. H2 is consequently only partially supported: the monotonic relationship is clear at fixed batch size but is not established as a global one-dimensional law.

X. EXPERIMENT 3: AUTHENTIC QWEN3 ROUTING AND CO-ACTIVATION

EXP-03 moves from synthetic routing to authentic physical routing traces from Qwen3-30B-A3B. The workload split reveals an important difference. GSM8K has an active working set averaging roughly 10.5–15.0 experts per layer across batch sizes, so a 32-expert fast tier can hold almost the complete

TABLE V
REPRESENTATIVE EXP-01 CAPACITY-CONSTRAINED RESULTS

Batch B	C/N	TierMoE hit-rate gain vs. Single (pp)	CXL traffic reduction (%)	Regime interpretation
8	0.25	+0.27	11.15	Moderate capacity pressure
16	0.25	+5.02	20.44	Severe capacity pressure
32	0.25	+8.68	13.20	Severe capacity pressure
32	0.50	+1.18	22.98	Moderate capacity pressure
All constrained conditions		+3.79 (mean)	16.94 (mean)	Paired test: $t = 6.42, p < 0.001$

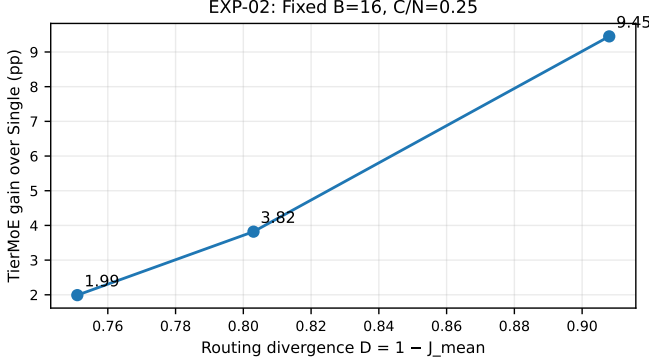


Fig. 3. EXP-02: TierMoE hit-rate advantage increases with routing divergence at fixed $B = 16$, while pooled analysis is confounded by working-set expansion.

active set. ShareGPT is much more dispersed: the working set is 36.2 experts at $B = 8$, 53.5 at $B = 16$, and 73.2 at $B = 32$, making $C = 32$ a meaningful capacity constraint.

Static LFU illustrates the problem. On ShareGPT at $B = 8$, $C = 32$, LFU achieves only 37.04% hit rate, whereas TierMoE reaches 92.51%, a 55.47 pp gap. The result reflects the difference between global popularity and immediate batch demand: conversational topics change, while a fixed global ranking cannot react to the active request composition.

The experiment also evaluates H3 using TierMoE-Batch-Aware-CoActivation. The co-activation variant maintains an exponential moving average of pairwise expert co-occurrence. In the evaluated comparison, greedy batch frequency reaches 92.51% hit rate with 2,663,680 MB modeled traffic, while co-activation reaches 92.23% with 2,648,832 MB. The paired hit-rate test gives $t = -1.387$, $p = 0.259$, so the small hit-rate difference is not statistically significant. Co-activation also takes 642.2 μs per step versus 51.2 μs for greedy, approximately 12.5 $\times$ slower.

Thus H3 is not supported. This negative result is useful because it narrows the design space: in the tested autoregressive traces, instantaneous batch demand is a more direct signal than an EMA of historical pairwise relationships.

XI. EXPERIMENT 4: COMPREHENSIVE BASELINE EVALUATION

EXP-04 evaluates TierMoE against the implemented baseline matrix on 24 constrained ShareGPT conditions. Table VI shows representative rows. The HBM-only upper bound remains 100%, while static LFU degrades severely as the active working set expands. At $B = 32$, $C = 32$, LFU reaches 38.70%,

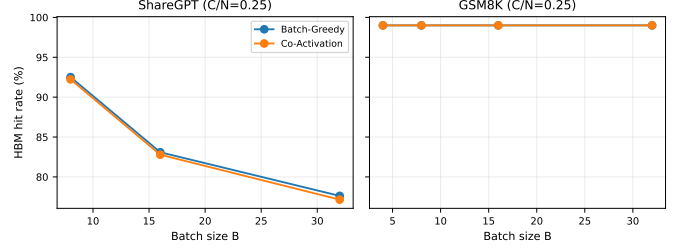


Fig. 4. EXP-03 authentic Qwen3 routing. GSM8K remains near the high-hit regime because its working set is small, whereas ShareGPT exposes the capacity contention that differentiates batch-aware placement.

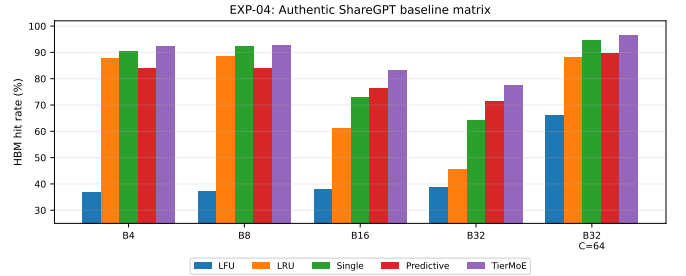


Fig. 5. EXP-04 baseline comparison on authentic ShareGPT routing. TierMoE retains the highest hit rate among the implemented constrained policies across the representative batch sizes.

reactive LRU 45.35%, Single-Request 64.20%, predictive 71.54%, and TierMoE 77.62%.

Across the 24 constrained rows, TierMoE improves hit rate by 6.99 pp over the predictive heuristic ($t = 13.58$, $p = 0.00086$) and by 16.61 pp on average over reactive LRU, with a maximum row-level margin of 32.27 pp. At $B = 16$, TierMoE improves over Single-Request by 10.23 pp; at $B = 32$ the margin is 13.42 pp. The broader comparison supports RQ4 for the implemented baselines.

The mechanism behind reactive LRU's degradation is intra-batch competition. Requests execute in micro-waves. If one request causes an expert to be evicted to load another expert, another request in the same step may immediately require the evicted block. A reactive policy responds after the conflict has occurred. TierMoE instead evaluates the aggregate demand before dispatch and attempts to keep high-utility experts resident throughout the batch step.

XII. CXL INTERCONNECT MODELING AND VALIDATION

Physical CXL Type-3 hardware was not available on the testbed. Therefore, system-scale CXL timing is intentionally

TABLE VI
REPRESENTATIVE EXP-04 RESULTS ON AUTHENTIC SHAREGPT TRACES

B	C	W	HBM-only	LFU	LRU	Single	Predictive	TierMoE
4	32	23.3	100.00	36.80	87.92	90.21	83.84	92.15
8	32	36.2	100.00	37.04	88.33	92.18	84.05	92.51
16	32	53.5	100.00	37.94	61.19	72.86	76.48	83.09
32	32	73.2	100.00	38.70	45.35	64.20	71.54	77.62
32	64	73.2	100.00	66.14	88.21	94.43	89.49	96.30

separated into a first-order aggregate bandwidth model and a discrete-event characterization of queue behavior using upstream CXLMemSim.

A. First-Order Bandwidth Model

For a serialized link carrying total parameter volume V_{total} , the first-order makespan is

$$T_{\text{link}} \approx \frac{V_{\text{total}}}{BW_{\text{CXL}}}. \quad (10)$$

A 256 MiB block at 32 GB/s therefore takes

$$\frac{268,435,456}{32 \times 10^9} = 8.3886 \text{ ms}. \quad (11)$$

A 300 ns read-latency term is negligible compared with this bulk transfer duration. The aggregate model is consequently used for EXP-05A and for converting traffic differences into modeled transfer-time differences. It is not presented as an end-to-end physical CXL latency measurement.

B. CXLMemSim Queue Characterization

EXP-05B investigates the upstream CXLMemSim endpoint implementation under one tested configuration: 32 GB/s, 300 ns read latency, a credit-based endpoint, a queue bound of 64, and two initial credits. The native C++ harness directly invokes the endpoint queue path rather than merely reproducing the equations in Python.

The endpoint uses a shared queue and credit state. With a 32 GB/s injection cadence of 2 ns per 64-byte cache line, short bursts exhibit substantial startup and queue effects. As the stream grows, these effects amortize. As shown in Fig. 6, empirical scaling over 64–262,144 cache lines follows approximately a $1/N$ convergence pattern. A fitted request-unit constant is around 5719 ± 68 , corresponding to the empirical startup characterization

$$T_0 = C\Delta t \approx 5719(2 \text{ ns}) = 11.44 \mu\text{s}. \quad (12)$$

At the 256 MiB scale (4,194,304 cache lines), the characterized stall factor approaches approximately 1.0014 under the tested injection and queue configuration.

Batch activation microbenchmarks over $K = 1, \dots, 32$ concurrent expert streams showed approximately one startup-overhead term for shared bursts in that specific simulator configuration. Model B, $V/BW + T_0$, reproduced the measured microbenchmark makespan with less than 0.4% error in the reported held-out checks, whereas a KT_0 model over-predicts shared-link overhead. These results should be interpreted as simulator characterization, not as proof that physical CXL links have a universal T_0 or that all stream organizations are equivalent.

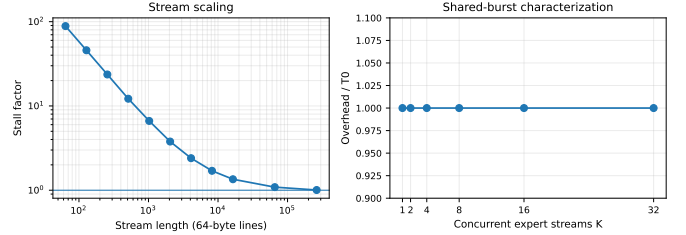


Fig. 6. EXP-05B native CXLMemSim characterization. Stream-level startup effects amortize with transfer length; batch-level measurements support a single-overhead model only under the tested queue and injection configuration.

XIII. EXPERIMENT 5: CXL SENSITIVITY ANALYSIS

EXP-05A evaluates 81 modeled conditions across bandwidths of 16, 32, and 64 GB/s and read-latency settings of 150, 300, and 600 ns. The objective is to determine whether the policy’s traffic reduction remains meaningful as the slow tier changes.

Bandwidth is the dominant first-order parameter. Scaling from 64 to 16 GB/s increases modeled transfer time by approximately $4\times$. In contrast, changing read latency from 150 to 600 ns changes the modeled makespan by less than 0.005% in the bulk-transfer regime. This indicates that the 256 MiB block granularity strongly amortizes per-access latency in the aggregate model.

For the authentic ShareGPT trace at 32 GB/s, the Single-Request and TierMoE transfer counts are 10,990 versus 10,405 at $B = 8$, 8,724 versus 8,263 at $B = 16$, and 6,242 versus 6,146 at $B = 32$. These correspond to modeled transfer-time reductions of 5.32%, 5.28%, and 1.53%, respectively, with a mean reduction of 4.05% across the three batch sizes. At lower bandwidth, the same byte-volume reduction produces larger absolute time savings. Compared with static LFU, TierMoE can generate more transfer activity because LFU freezes its cache; the trade-off is a much higher hit rate. For example, at $B = 8$, $C = 32$, LFU achieves 37.04% while TierMoE reaches 92.51%.

XIV. OVERALL RESULTS AND DISCUSSION

Table VIII summarizes the six research axes. The central result is consistent across synthetic and authentic routing: the value of batch awareness emerges when the aggregate working set exceeds the fast-memory quota. TierMoE uses cross-request overlap as a direct placement signal and avoids requiring a long history or per-sequence prediction model.

TABLE VII
EXP-05A TRANSFER COUNTS AND MODELED TIMES AT 32 GB/s

B	Policy	Transfers	Traffic (MB)	T_{link} (s)	Reduction vs. Single
8	Single	10,990	2,813,440	92.191	—
8	TierMoE	10,405	2,663,680	87.283	5.32%
16	Single	8,724	2,233,344	73.182	—
16	TierMoE	8,263	2,115,328	69.315	5.28%
32	Single	6,242	1,597,952	52.362	—
32	TierMoE	6,146	1,573,376	51.556	1.53%

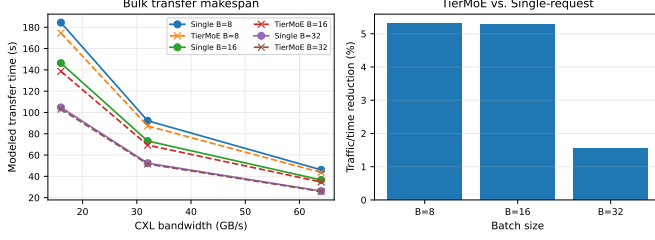


Fig. 7. EXP-05A modeled CXL sensitivity. Transfer time scales approximately inversely with bandwidth, while changing the read-latency parameter has a negligible first-order effect at 256 MiB block granularity.

TABLE VIII
OVERALL SYNTHESIS

Axis	Main measurement	Verdict
Concurrency	+5.02–8.68 pp; 13.20–20.44% traffic	Supported under pressure
Divergence	+1.99–9.45 pp at fixed $B = 16$	Partially supported
Co-activation	–1.25 pp, $p = .259$; $12.5\times$ overhead	Not supported
Baselines	+6.99 pp vs predictive; up to +32.27 pp vs LRU	Supported for implemented baselines
CXL sensitivity	$4\times$ bandwidth scaling; latency $< 0.005\%$	Supported as modeled sensitivity
CXLMemSim	$T_0 \approx 11.44 \mu\text{s}$; $< 0.4\%$ microbenchmark error	Supported for tested configuration

A. Why Batch-Aware Selection Helps

The placement utility of a fast-memory slot increases when the corresponding expert is needed by several requests in the same step. Single-request placement can spend capacity on an expert useful to one sequence while ignoring another expert needed by several concurrent sequences. Under capacity pressure, this reduces the number of requests whose immediate demands are resident. TierMoE effectively amortizes a fast-memory slot across the concurrent batch.

B. Why Historical Co-Activation Did Not Help

The co-activation experiment provides a useful counterexample to the assumption that more historical structure is always better. Autoregressive routing can change quickly with the current token context. An EMA retains past pairwise relationships after the active demand has moved elsewhere. The instantaneous histogram has no such lag. In this workload, the extra state and computation therefore did not translate into statistically significant residency gains.

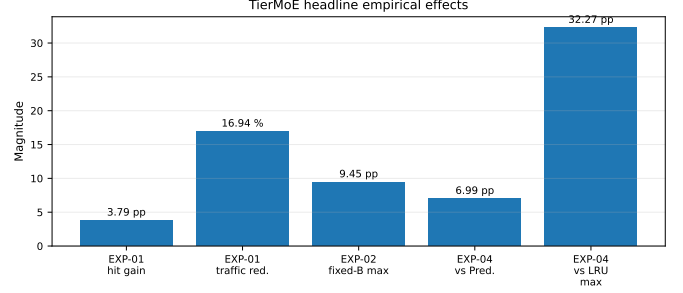


Fig. 8. Summary of the main empirical effects across the study. The strongest improvements occur under capacity pressure, while the CXL study shows that bandwidth dominates the first-order transfer model.

C. Traffic Versus Hit Rate

Minimizing CXL traffic is not identical to maximizing fast-memory hit rate. Static LFU can avoid dynamic promotions and therefore appear attractive under a traffic-only objective, but it can produce very poor residency for dynamic workloads. TierMoE deliberately trades some migration activity for a much higher probability that currently executing experts are resident. The appropriate system objective is therefore closer to minimizing memory-stall cost than minimizing bytes alone.

D. Solver Cost and Serving Context

The $51.2 \mu\text{s}$ solver overhead is small compared with a single 256 MiB modeled transfer at 32 GB/s. The co-activation variant demonstrates that a more complex placement signal can consume much more CPU time without measurable benefit. The present study does not claim that solver cost is negligible in every serving architecture; interaction with GPU kernel overlap, request arrival dynamics, and end-to-end tail latency remains to be measured.

XV. LIMITATIONS AND THREATS TO VALIDITY

Several limitations define the scope of the conclusions:

- 1) **No physical CXL hardware.** Experiments use dual RTX A6000 GPUs; CXL behavior is modeled and characterized in simulation rather than measured on a CXL ASIC or physical memory expander.
- 2) **First-order system timing.** System-scale CXL transfer timing primarily uses V/BW , with simulator-derived startup behavior analyzed separately.
- 3) **Simulator configuration scope.** CXLMemSim results correspond to the tested 32 GB/s, 300 ns, credit-based configuration and should not be generalized to every CXL controller.

- 4) **Representative native simulation.** Full-scale native queue simulation of every 64-byte cache line for every expert transfer was not executed; the study uses bounded microbenchmarks and bulk modeling.
- 5) **Conceptual baselines.** Predictive lookahead and reactive CXL-LRU are trace-driven algorithmic approximations inspired by prior paradigms, not full native reproductions.
- 6) **One model family.** Authentic evaluation focuses on Qwen3 with 128 experts and top-8 routing. Other MoE architectures may route differently.
- 7) **Controlled synthetic routing.** Synthetic distributions provide experimental control but do not exhaustively represent production multi-tenant workloads.
- 8) **Fixed expert blocks.** Experts are transferred as 256 MiB blocks; finer parameter slicing was not studied.
- 9) **Read-only weights.** Weight writeback is assumed to be zero during inference.
- 10) **Static fast-tier quotas.** Expert capacity is evaluated using fixed quotas rather than joint dynamic allocation against growing KV caches.
- 11) **No end-to-end CXL serving latency.** Overlap between CXL transfers and GPU computation, request queuing, throughput, and tail latency were not measured on a live physical CXL serving stack.

XVI. FUTURE WORK

The most direct extensions follow from these limitations. First, a physical CXL 2.0/3.0 testbed would permit direct measurement of transfer overlap, controller contention, and multi-expander topology effects. Second, expert weights and paged KV cache should be co-allocated in the fast tier so that placement responds to sequence context length as well as routing demand. Third, asynchronous expert promotion could overlap CXL movement with attention and preceding-layer computation. Fourth, the hysteresis parameter λ could be tuned online according to observed routing switching rates.

Further directions include combining lightweight sequence prediction with batch aggregation, evaluating larger MoE architectures with hundreds of billions of parameters, measuring end-to-end serving throughput and tail latency, and extending placement to topology-aware multi-link CXL fabrics. These directions also provide a path toward replacing the first-order bulk model with a validated end-to-end memory-system model.

XVII. CONCLUSION

This paper presented TierMoE, a batch-aware expert-placement framework for memory-tiered MoE inference. The framework aggregates immediate expert demand across concurrent requests and uses marginal utility with residency hysteresis to select a capacity-constrained HBM resident set. Across controlled synthetic experiments and 461,184 authentic Qwen3 routing decisions, the results show that cross-request aggregation is most valuable when concurrent working sets exceed fast-memory capacity. Representative constrained operating points achieve 5.02–8.68 pp higher hit rate and 13.20–20.44% lower modeled CXL parameter traffic than isolated placement, while broader baseline comparisons show 6.99 pp

improvement over the implemented predictive heuristic and up to 32.27 pp over reactive LRU.

The study also identifies two important boundaries. Pairwise co-activation history did not significantly improve residency over instantaneous batch frequency and imposed substantially higher solver cost. Meanwhile, the CXL analysis shows that bulk bandwidth dominates first-order transfer timing for the chosen 256 MiB expert granularity. CXLMemSim characterization provides useful queue-level evidence under a specified configuration, but does not replace physical CXL measurements. Taken together, the findings support batch-aware placement as a practical systems mechanism and provide a reproducible foundation for future work on joint expert/KV tiering and physical CXL deployments.

ACKNOWLEDGMENT

The author would like to thank Avinash for providing compute access to a dual-NVIDIA RTX A6000 workstation, which enabled the model profiling and experimental evaluation presented in this work.

REFERENCES

- [1] A. Q. Jiang et al., “Mixtral of Experts,” arXiv:2401.04088, 2024.
- [2] DeepSeek-AI, “DeepSeek-V3 Technical Report,” arXiv:2412.19437, 2024.
- [3] Qwen Team, “Qwen2.5: A Party of Foundation Models,” arXiv:2412.15115, 2024.
- [4] S. Rajbhandari et al., “DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Unprecedented Scale,” in Proc. ICML, 2022.
- [5] F. Fiddler et al., “Fiddler: Serving Large MoE Models on a Single GPU with CPU Offloading,” in Proc. EuroSys, 2024.
- [6] L. Fu et al., “MoE-Infinity: Fast MoE Serving with Activation-Aware Expert Offloading,” in Proc. USENIX OSDI, 2024.
- [7] X. ProMoE et al., “ProMoE: Fast, Memory-Efficient MoE Serving via Predictive Expert Prefetching,” in Proc. ACM ASPLOS, 2025.
- [8] Y. CXL-MoE et al., “CXL-MoE: Accelerating Mixture-of-Experts LLM Serving via CXL Memory Pooling and Near-Data Processing,” in Proc. IEEE/ACM ISCA, 2023.
- [9] Z. HybriMoE et al., “HybriMoE: Hybrid Memory Scheduling for Heterogeneous MoE Inference,” in Proc. IEEE MICRO, 2024.
- [10] K. FIRM-MoE et al., “FIRM-MoE: Fine-Grained Memory Management for CXL-Based MoE Inference,” in Proc. IEEE HPCA, 2024.
- [11] CXLMemSim Team, “CXLMemSim: A Discrete-Event Simulator for CXL Memory Expanders,” GitHub repository, 2023.
- [12] CXL Consortium, “Compute Express Link Specification, Revision 3.0,” 2022.
- [13] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” in Proc. ICLR, 2017.
- [14] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” JMLR, vol. 23, no. 120, 2022.
- [15] L. Zheng et al., “Efficiently Scaling LLM Serving with Structured Generation and SGLang,” in Proc. USENIX OSDI, 2024.

APPENDIX A

EXPERIMENTAL CONFIGURATION MATRIX

Table IX summarizes the principal project phases.

APPENDIX B

MATHEMATICAL BASELINE FORMULATIONS

Static LFU computes global access frequency

$$F(e) = \sum_{t=1}^T f_e(t) \quad (13)$$

TABLE IX
EXPERIMENTAL CONFIGURATION MATRIX

Phase	Workload	Policies	Parameter sweep	Sample
EXP-01	Synthetic, Zipf $\alpha = 1.0$	TierMoE, Single, LFU, Naive	$B = 1-32$; $C/N = 0.25-1.0$	432×3 seeds
EXP-02	Synthetic clustered	TierMoE, Single	$\alpha = 0.8-1.4$; $B = 4-32$	384×3 seeds
EXP-03	Authentic Qwen3	Greedy, CoAct, Single, LFU	$B = 4-32$; $C = 32, 64$	461,184 events
EXP-04	Authentic ShareGPT	TierMoE, Predictive, LRU, LFU	$B = 4-32$; $C = 32, 64$	24 rows
EXP-05A	Modeled CXL	TierMoE, Single, LFU	BW=16–64 GB/s; latency=150–600 ns	81 conditions
EXP-05B	CXLMemSim queue	Model B/C and native queue characterization	$N = 64-262,144$; $K = 1-32$	75,546 discrete workload transfers represented by bulk model

and pins the top- C experts by $F(e)$. Single-Request uses

$$S_{\text{single}}(e) = \mathbf{1}\{e \in \mathcal{D}_1\} + \lambda \mathbf{1}\{e \in \mathcal{M}_{\text{current}}\}. \quad (14)$$

The predictive heuristic maintains per-sequence activation probability

$$P_e^{(s)}(t+1) = \gamma P_e^{(s)}(t) + (1-\gamma) \mathbf{1}\{e \in \mathcal{D}_s(t)\}, \quad (15)$$

with $\gamma = 0.85$. Reactive LRU stores an access timestamp $\tau(e)$ and evicts

$$e_{\text{evict}} = \arg \min_{e \in \mathcal{M}_{HBM}} \tau(e) \quad (16)$$

on an expert miss.

- `src/workload/generator.py`: synthetic Zipf and clustered trace generation;
- `calibration/stream_scaling_bench.cpp`: native CXLMemSim stream scaling benchmark;
- `calibration/batch_activation_validation.cpp`: native batch activation harness;
- `results/`: JSON experimental artifacts for EXP-01 through EXP-05B;
- `analysis/`: plotting, statistical validation, and report-generation pipelines.

APPENDIX C

AUTHENTIC TRACE CHARACTERISTICS

GSM8K exhibits concentrated routing, with active working sets of approximately 10.5 ± 1.8 experts at $B = 4$ through 15.0 ± 2.1 at $B = 32$. With $C = 32$, the active set therefore fits in the fast tier and dynamic policies remain above 99% hit rate. ShareGPT instead reaches 36.2 ± 4.2 experts at $B = 8$, 53.5 ± 5.6 at $B = 16$, and 73.2 ± 6.8 at $B = 32$, creating the capacity contention that exposes the difference between static and batch-aware policies.

APPENDIX D

CXLMEMSIM DISCRETE-EVENT QUEUE DETAILS

The tested endpoint uses a bounded input queue, initial credits, and an in-flight request structure. At 32 GB/s, the modeled 64-byte arrival cadence is 2 ns. The pipeline includes frontend, forwarding, memory-read, response, and protocol components totaling approximately 351.5 ns before additional congestion effects. The observed startup characterization is amortized as streams become long. The resulting $T_0 \approx 11.44 \mu\text{s}$ and $S \approx 1.0014$ bulk stall factor are properties of this tested simulator configuration, not universal CXL constants.

APPENDIX E

REPRODUCIBILITY AND CODEBASE MAPPING

The prototype repository separates algorithm implementations, routing profiling, workload generation, native CXL calibration, results, and analysis:

- `src/algorithm/`: TierMoE greedy solver and LFU, LRU, predictive, and single-request baselines;
- `src/profiler/router_hook.py`: non-intrusive Qwen routing hooks;